Практическая работа № 2 (12.10)

Разработка REST API на FastAPI с использованием PostgreSQL, SQLAlchemy и Pydantic

Цель работы: получить навыки разработки веб-сервиса на FastAPI с хранением данных в PostgreSQL через ORM SQLAlchemy 2.0, валидации данных средствами Pydantic v2 и использования механизмов Starlette (middleware).

1. Требования к окружению

- Python 3.11 или новее
- Docker и Docker Compose
- Git, аккаунт на GitHub
- IDE (VS Code или PyCharm)

2. Постановка задачи

Разработать сервис «Менеджер задач». Сервис хранит пользователей и их задачи и предоставляет REST API для работы с ними.

Сущности:

Сущность	**Поля**
User	id (PK), email (уникальный), name, created_at
Task	id (PK), title (1-200 символов), description (необязательно), done (bool, по умолчанию false), created_at, owner_id (FK User)

Связь: один пользователь - много задач.

Обязательные эндпоинты:

Метод	**Путь**	**Описание**	**Коды ответа**
POST	/users	создать пользователя	201, 409 (email занят), 422
GET	/users/{id}	получить пользователя	200, 404
POST	/tasks	создать задачу	201, 404 (нет владельца), 422
GET	/tasks	список задач с фильтрами done, owner_id и пагинацией limit (1-100), offset	200
GET	/tasks/{id}	получить задачу	200, 404
PATCH	/tasks/{id}	частично обновить задачу	200, 404, 422
DELETE	/tasks/{id}	удалить задачу	204, 404

3. Порядок выполнения

Шаг 1. Создание репозитория и структуры проекта.

Создайте репозиторий со следующей структурой:

…

```
├── app/
│   ├── __init__.py
│   ├── main.py         # создание приложения, подключение
роутеров
│   ├── config.py       # настройки (pydantic-settings)
│   ├── db.py           # движок, сессия, Base
│   ├── models.py       # ORM-модели
│   ├── schemas.py      # Pydantic-схемы
│   └── routers/
│       ├── users.py
```

```
│       └── tasks.py
├── docker-compose.yml
├── requirements.txt
├── .env.example
└── README.md
```

Шаг 2. Запуск PostgreSQL. Создайте docker-compose.yml:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: tasks
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
volumes:
  pgdata:
```

Запустите базу командой docker compose up -d db и проверьте её доступность: docker compose exec db psql -U app -d tasks -c "select 1".

Шаг 3. Установка зависимостей. Файл requirements.txt:

```
fastapi
uvicorn[standard]
sqlalchemy>=2.0
asyncpg
pydantic>=2
pydantic-settings
alembic
```

Шаг 4. Конфигурация. Строку подключения к БД нельзя записывать в коде. Её нужно читать из переменной окружения DATABASE_URL через pydantic-settings.

Шаг 5. Подключение к БД. В db.py создайте асинхронный движок, фабрику сессий и зависимость get_session, которая отдаёт сессию через yield.

Шаг 6. ORM-модели. Опишите модели в стиле SQLAlchemy 2.0, то есть через Mapped и mapped_column. Связь оформите через relationship с параметром back_populates.

Шаг 7. Pydantic-схемы. Для каждой сущности создайте отдельные схемы *Create, *Read и *Update. В схемах *Update все поля необязательны. В схемах *Read должно быть включено from_attributes=True. Для email используйте тип EmailStr.

Шаг 8. Роутеры. Реализуйте эндпоинты через APIRouter. Сессию получайте только через Depends(get_session). Ошибки возвращайте через HTTPException с кодами из таблицы выше.

Шаг 9. Миграции. Инициализируйте Alembic командой alembic init -t async migrations. Создайте первую миграцию с --autogenerate и примените её. Таблицы не должны создаваться через create_all.

Шаг 10. Middleware (Starlette). Добавьте middleware, которое:

- добавляет в ответ заголовок X-Process-Time;
- пишет в лог метод, путь, статус и время обработки запроса.

Шаг 11. Проверка. Запустите сервис командой uvicorn app.main:app --reload, откройте /docs и выполните все запросы через Swagger UI.

4. Требования к отчёту

В README.md репозитория должны быть:

- Инструкция по запуску (команды по шагам).
- Скриншоты Swagger UI с успешными запросами и ответом 404.
- Скриншот таблиц в БД (\dt в psql).
- Ответы на контрольные вопросы.

5. Контрольные вопросы

- Чем FastAPI отличается от Starlette и какие компоненты Starlette он использует?
- Зачем разделять схемы Create, Read и Update?
- Что делает from_attributes=True?
- Почему сессия передаётся через Depends, а не создаётся глобально?
- Чем exclude_unset=True отличается от exclude_none=True?
- Почему в production таблицы не создают через create_all?